Using fixtures instead

The Django `TestCase` class has an attribute named `fixtures` that takes a list of fixture filenames to load. You can use this instead of explicitly creating content. Fixture data gets loaded into the database once, at class creation time. Django will look for fixtures in your app directories as well as anywhere named in the `FIXTURE_DIRS` list in settings.py.

I prefer not to use fixtures in my tests; explicitly creating the objects in the test feels clearer to me. I also don't like that there is no way of separating test fixtures from app fixtures and that it is easy to accidentally load your test data into the real database.

8.4 Expecting errors

You might be feeling pretty good about your test for the musician view, but you still haven't covered all possible scenarios. The view does a look-up of a `Musician` object by ID and is supposed to raise a `404` error if the `Musician` does not exist. You want to test that this code works as well. A good test ensures that negative cases get handled properly, instead of crashing the code.

Typically in a Python script, if an exception is raised, the program exits and a stack trace is displayed, unless the code catches the exception. The musician view raises an `Http404` error, but that doesn't exit the program because Django catches it. Django's error handling for this situation results in a `404` error page being presented to the user. A `404` error page sets the HTTP status code to `404`, hence the name.

Testing that a view raises an `Http404` error is a matter of validating that a status code of `404` is returned from the call. This is rather similar to checking a normal case, with a different expected return value. Listing 8.3 adds the new scenario to the test class, using an ID of `10` for the call, which doesn't exist in the database. Since a new test database is created each time you run your test suite, you know an ID of `10` doesn't exist as long as you have created fewer than 10 musicians in the test scenario. If you wanted to be really sure, you could query the ID of the last `Musician` (`Musician.objects.last().id`) and add `1` to its value, but I've simply hardcoded a bigger number instead.

Listing 8.3 Testing that an error gets handled properly

```
# bands/tests.py
...

class TestBands(TestCase):
    ...
    def test_musician_404(self):
        url = "/bands/musician/10/"
        response = self.client.get(url)
        self.assertEqual(404, response.status_code)
```

Look up the non-existent musician with ID 10.

Check for the 404 response code.

Although Django wraps a `404` exception, there are scenarios in which you want to test uncaught exceptions. You likely won't need to do this in a view, but you may want to